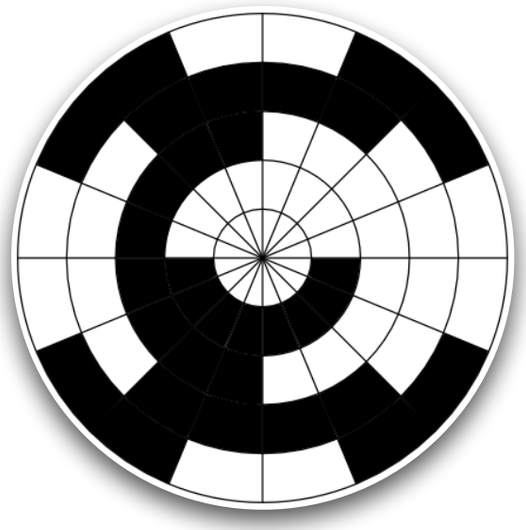
# Enumeration

## Combinatorial Algorithms

Valentin Cassano

Guangdong Technion Israel Institute of Technology

# Subsets



Gray Code

Given $A, B : 2^{[0:n)}$, the symmetric difference of $A$ and $B$ is

$$A \ominus B = (A \cup B) \setminus (A \cap B)$$

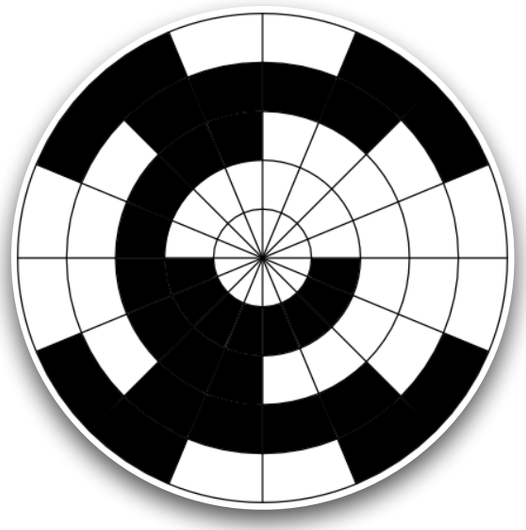
On this basis, we define the distance between $A$ and $B$ as

$$\mathrm{dist}(A, B) = \#(A \ominus B)$$

In brief, $\mathrm{dist}(A, B)$ indicate in how many entries $\chi(A)$ and $\chi(B)$ differ from each other.

The function $\mathrm{dist}$ is called the Hamming distance. Intuitively, this distance tells us how different these sets are from each other: the greater the distance, the more different.

# Subsets



Gray Code

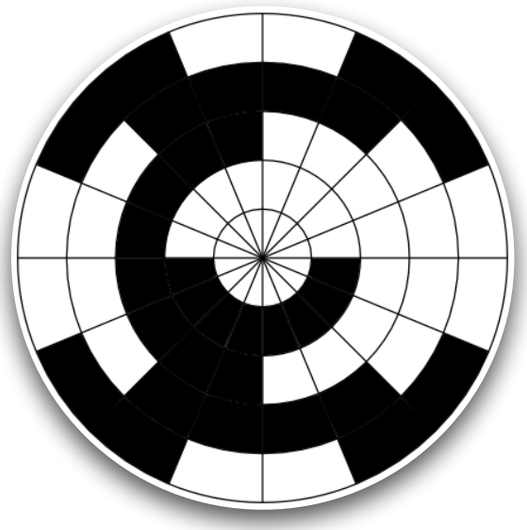
The lexicographic order on subsets yields very simple ranking and unranking algorithms.

This ordering, however, is not well suited for generating sequentially all $2^n$ subsets of in $2^{[0:n)}$. The reason is that subsets that are relatively consecutive in the ordering can be very “different.”

For instance, in $2^{[0:3)}$, we get $\mathbf{rank}(\{0,1\}) = 3$ and $\mathbf{rank}(\{2\}) = 4$. These two consecutive subsets have $\mathbf{dist}(\{0,1\}, \{2\}) = 3$, that is the maximum possible distance, indicating they share no elements at all.

In generating subsets sequentially, we wish to do so choosing consecutive subsets with the smallest possible Hamming distance. This would enable us to obtain any subset from its predecessor by adding or deleting a single element. We refer to such an ordering of subsets as a minimal change ordering.

# Subsets



Gray Code

The following linear arrangement of the subsets in $2^{[0:3)}$ is an example of a minimal change ordering

$$\varnothing, \{0\}, \{0,1\}, \{1\}, \{1,2\}, \{0,1,2\}, \{0,2\}, \{2\}$$

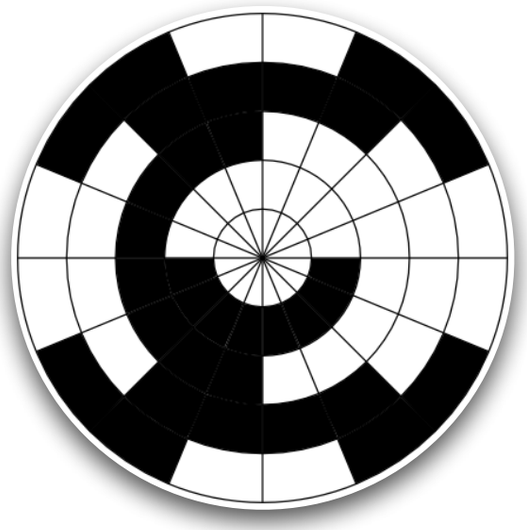
The characteristic vectors of the subsets in a minimal change ordering form a structure that is known as a Gray code.

Precisely, a Gray code is an ordering of all characteristic vectors in such a way that the Hamming distance for any two consecutive vectors is one.

For example, for the minimal change ordering above, we obtain the following Gray code:

$$000, 001, 011, 010, 110, 111, 101, 100$$

# Subsets

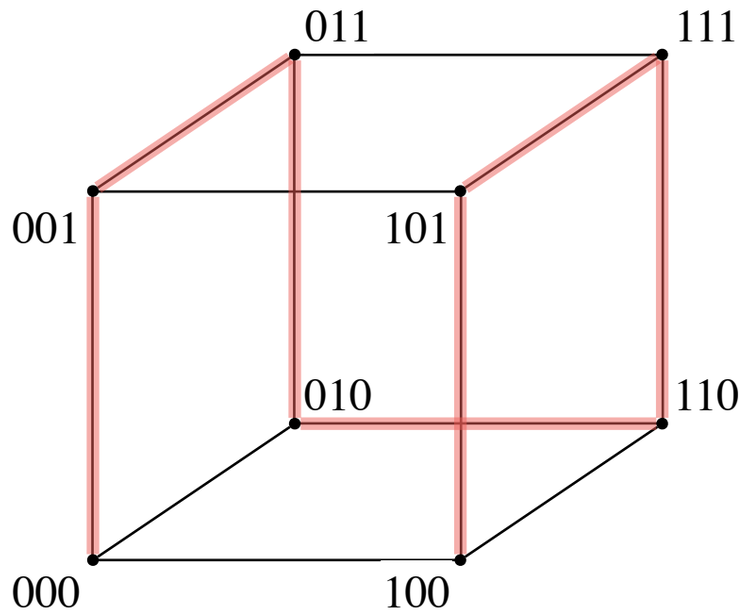


Gray Code

There is a very elegant geometric way to define what Gray codes are.

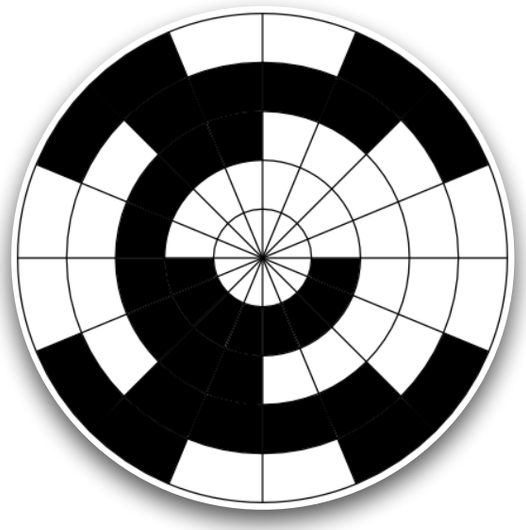
Consider the $n$-dimensional unit cube whose $2^n$ vertices are labeled by the binary vectors of length $n$, and whose edges join vertices having Hamming distance equal to one.

E.g.,




Then, a Gray code is a Hamiltonian path, that is, a way traversing the edges of the cube visiting each vertex exactly once.

# Subsets



Gray Code

There has been a considerable amount of study done on different constructions for Gray codes. We will look at a particularly nice class of Gray codes called (binary reflected) Gray codes.

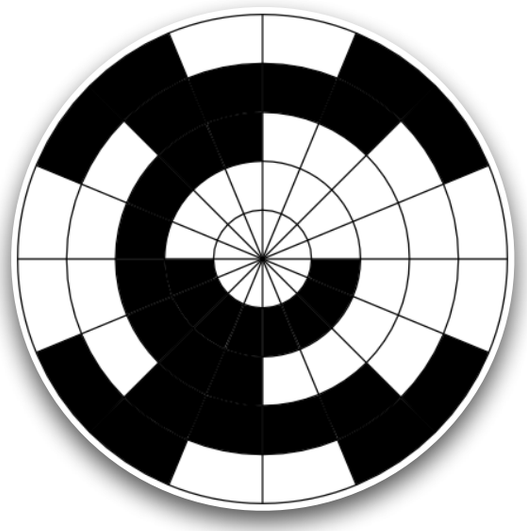
We begin by defining a list $\mathbf{G}^n$ inductively as:

$$\mathbf{G}^0 = [[]]$$
$$\mathbf{G}^{(n+1)} = 0 \triangleright \mathbf{G}^n +\!\!+ 1 \triangleright \text{rev}(\mathbf{G}^n)$$

where $x \triangleright X$ is the list obtained by appending $x$ at the beginning of all lists in $X$ and $\mathbf{rev}(\mathbf{G}^n)$ is the reverse of $\mathbf{G}^n$. The name reflected comes from the second copy of $\mathbf{G}^n$ being reversed.

The construction works because the first half $0 \triangleright \mathbf{G}^n$ ends on the same bit string as the second half $1 \triangleright \text{rev}(\mathbf{G}^n)$ begins, differing only in the leading bit, so the join between the two halves introduces exactly one bit-flip. By induction, $\mathbf{G}^n$ is a Gray code for all $n$.

# Subsets



Gray Code

Let us compute $\mathbf{G}^3$ from the inductive definition.

$$\mathbf{G}^0 = [\ [\ ]\ ]$$
$$\mathbf{G}^1 = 0 \triangleright \mathbf{G}^0 +\!\!+ 1 \triangleright \mathrm{rev}(\mathbf{G}^0) = [\ [0]\ ] +\!\!+ [\ [1]\ ] = [\ [0],\ [1]\ ]$$

$$\mathbf{G}^2 = 0 \triangleright \mathbf{G}^1 +\!\!+ 1 \triangleright \mathrm{rev}(\mathbf{G}^1)$$
$$= [\ [00],\ [01]\ ] +\!\!+ [\ [11],\ [10]\ ] = [\ [00],\ [01],\ [11],\ [10]\ ]$$

$$\mathbf{G}^3 = 0 \triangleright \mathbf{G}^2 +\!\!+ 1 \triangleright \mathrm{rev}(\mathbf{G}^2)$$

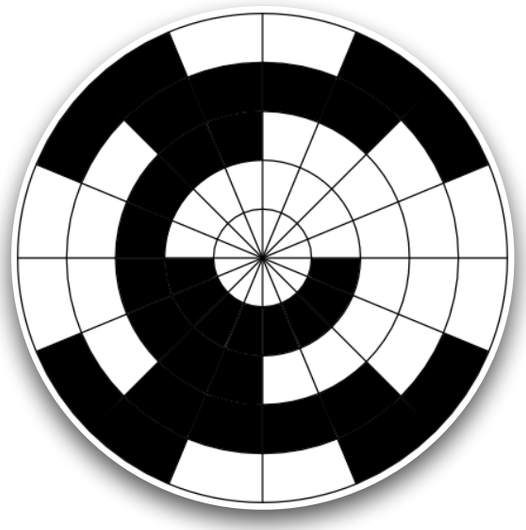
$$0 \triangleright \mathbf{G}^2 = [\ [000],\ [001],\ [011],\ [010]\ ]$$

$$1 \triangleright \mathrm{rev}(\mathbf{G}^2) = [\ [110],\ [111],\ [101],\ [100]\ ]$$

$$\mathbf{G}^3 = [\ 000,\ 001,\ 011,\ 010,\ 110,\ 111,\ 101,\ 100\ ]$$

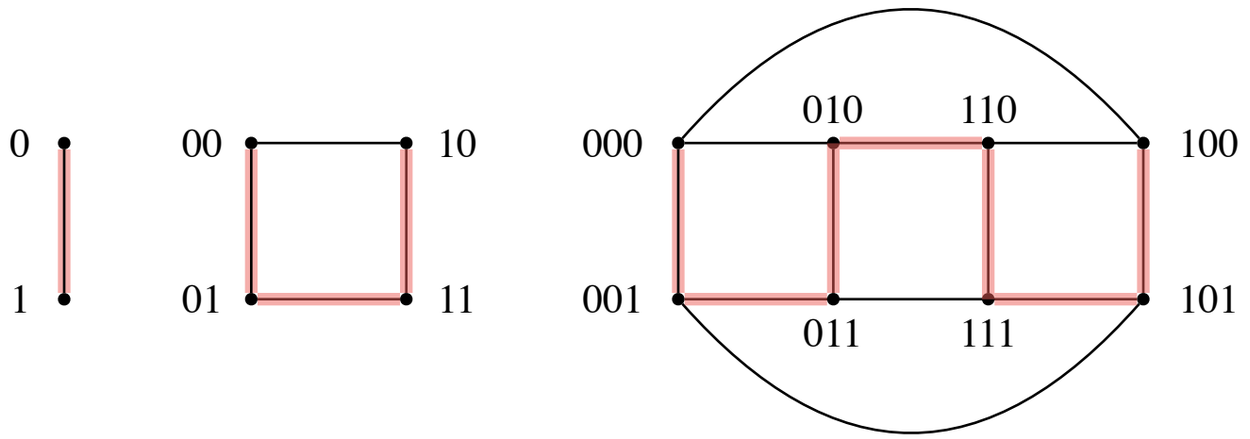
This matches the Gray code given on the minimal-change ordering slide. Each consecutive pair differs in exactly one bit; the join between the two halves also differs in exactly one bit, that is, the leading bit introduced by the reflect step.

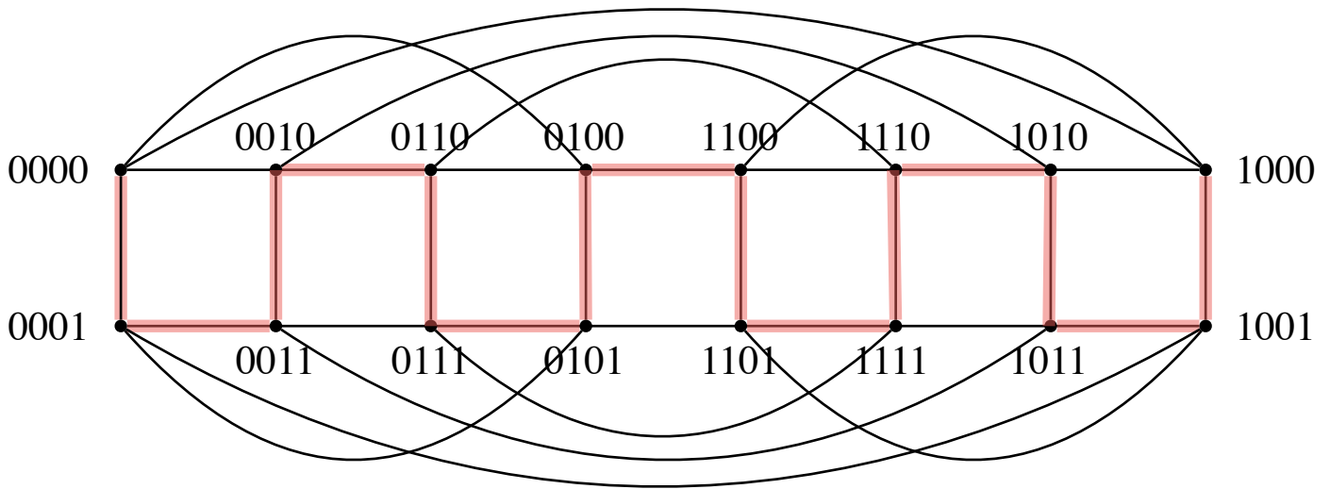
# Subsets



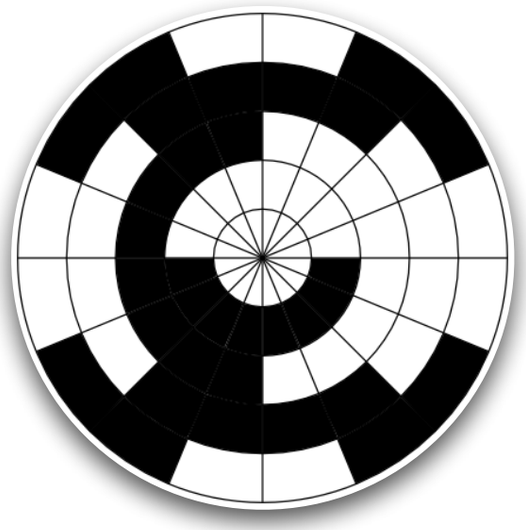
Gray Code

The evolution of reflected Gray codes.



# Subsets



Gray Code

Ranking and unranking algorithms for Gray codes take into account the following relations between $\mathbf{bin}(i)$ and $\mathbf{G}_i^n$:

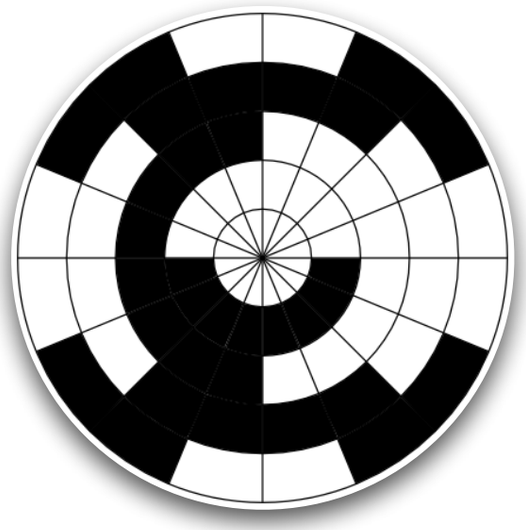
1. $\mathrm{bin}(i) = \bigoplus_{k=0}^{n-1} \mathrm{sh}(\mathbf{G}_i^n, -k)$ — rank
2. $\mathbf{G}_i^n = \mathrm{bin}(i) \oplus \mathrm{sh}(\mathrm{bin}(i), -1)$ — unrank

where: $\bigoplus$ is 'xor', and $\mathrm{sh}(X, -k) = ([0] \times k) +\!\!+ X[0{:}(\#X - k)]$.

For instance, notice that for $n = 3$ we have:

| $i$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| $\mathrm{bin}(i)$ | 000 | 001 | 010 | 011 | 100 | 101 | 110 | 111 |
| $\mathbf{G}_i^3$ | 000 | 001 | 011 | 010 | 110 | 111 | 101 | 100 |
| $A$ | 0 | {0} | {0,1} | {1} | {1,2} | {0,1,2} | {0,2} | {2} |

# SUBSETS



Gray Code

Let us work out an example of the rank and unrank formulas for $n = 3$.

unrank — Given $i = 5$, compute $\mathbf{G}_5^3 = 111$.

$$\text{bin}(5) = 101$$

$$\text{sh}(101, -1) = 010 \quad \text{(shift right by 1, padding with 0 on the left)}$$

$$\mathbf{G}_5^3 = 101 \oplus 010 = 111 \quad \Rightarrow \quad A = \{0,1,2\}\checkmark$$

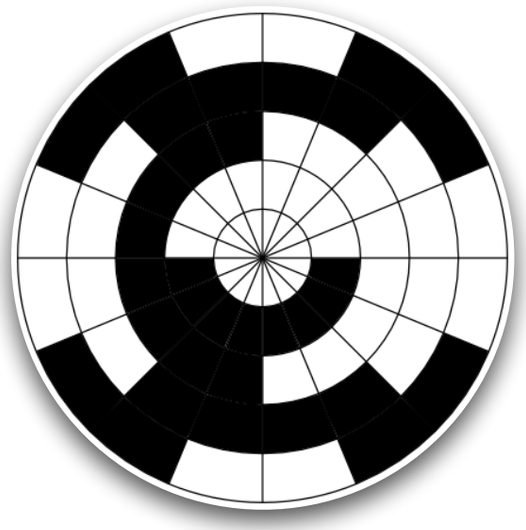
rank — Given $A = \{0,1,2\}$, so $\mathbf{G}_5^3 = 111$, recover $i = 5$.

$$\text{sh}(111,0) = 111$$
$$\text{sh}(111, -1) = 011$$
$$\text{sh}(111, -2) = 001$$

$$\text{bin}(i) = 111 \oplus 011 \oplus 001 = 101 \quad \Rightarrow \quad i = 5\checkmark$$

The two formulas are inverses: applying unrank then rank (or rank then unrank) returns the original input.

# Subsets



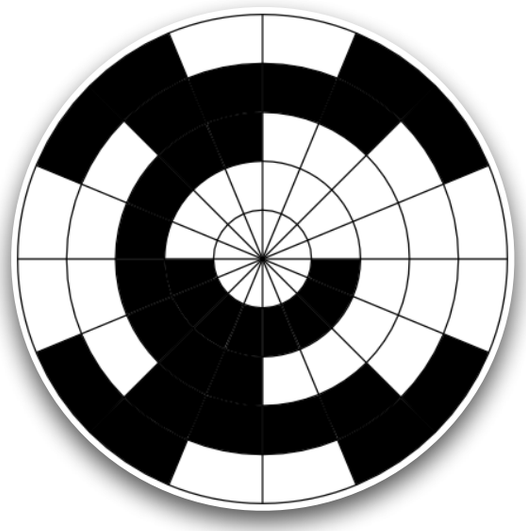
Gray Code

```
def rank(A: set, n: int) -> int:
    """
    an implementation of the rank algorithm
    """
    # require A: 2^[0:n)


    # to be completed in class


    return r
    # ensure: G^n_r = X(A)
```

# Subsets



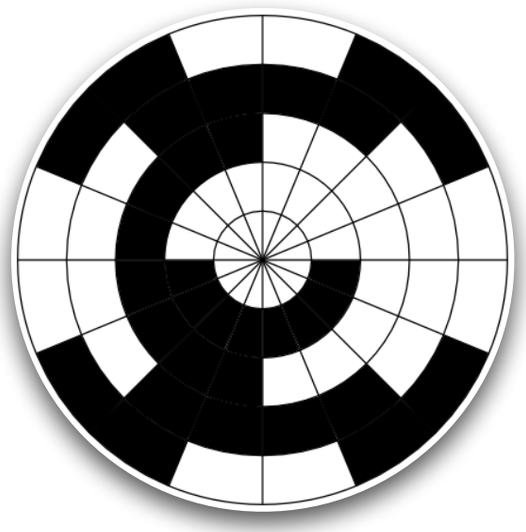
Gray Code

```
def unrank(r: int, n: int) -> set:
    """
    an implementation of the unrank algorithm
    """
    # require: r: [0:2^n)


    # to be completed in class


    return A
    # ensure: G^n_r = X(A)
```

# Subsets



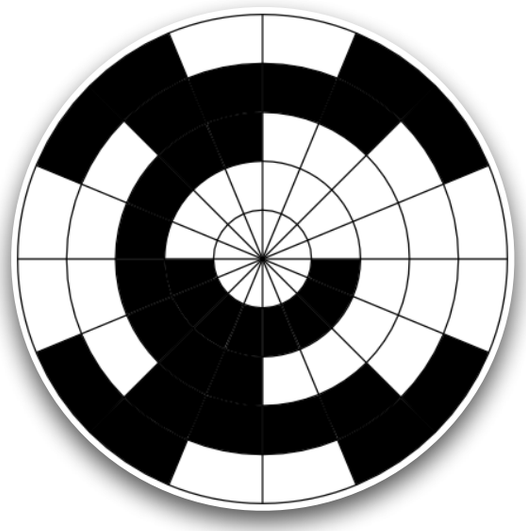
Gray Code

```
def successor(A: set, n: int) -> set:
    """
    an implementation of the successor algorithm
    """
    # require: A: 2^[0:n)


    # to be completed in class


    return B
    # ensure: rank(B) = rank(A)+1 (mod 2^n)
```

# Subsets



Gray Code

How to think about the successor algorithm.

The postcondition $\text{rank}(B) = \text{rank}(A) + 1 \pmod{2^n}$ suggests two routes to an implementation.

The naive approach is via rank and unrank (generic recipe):

$$\text{successor}(A) = \text{unrank}((\text{rank}(A) + 1) \bmod 2^n)$$

In this case, the algorithm becomes:

1. $r = \text{rank}(A)$ using the formula:

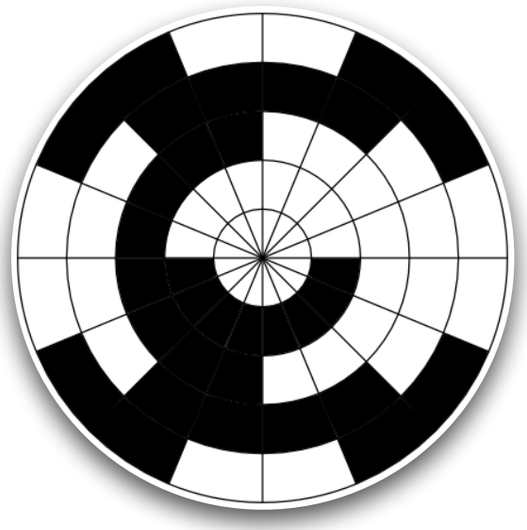
$$r = \text{nat}\left(\bigoplus_{k=0}^{n-1} \text{sh}(\chi(A), -k)\right)$$

2. increment: $r \leftarrow (r + 1) \bmod 2^n$
3. recover $B = \text{unrank}(r')$ using the formula:

$$\chi(B) = \text{bin}(r) \oplus \text{sh}(\text{bin}(r), -1)$$

This always works but performs two full passes over the $n$-bit vector: one to rank, one to unrank; for a total of $O(n)$ work with a constant that may be larger than necessary.

# Subsets



Gray Code

The alternative to the naive approach is to avoid using rank and unrank.

Recall that $\mathbf{G}^n$ is built by reflecting $\mathbf{G}^{n-1}$, and that moving from $\mathbf{G}_i^n$ to $\mathbf{G}_{i+1}^n$ flips exactly one bit.

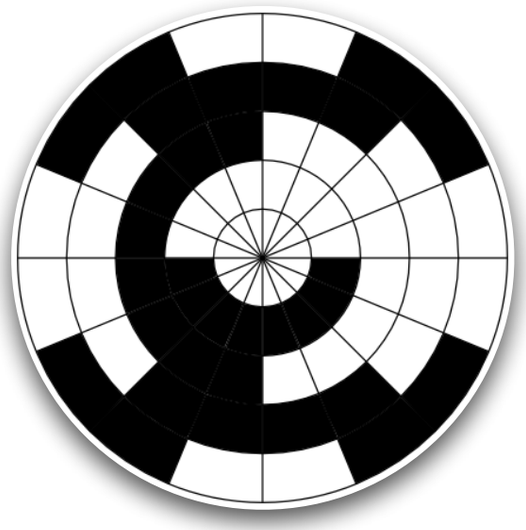
The question is: which bit to flip? The position of the lowest-order set bit in $\mathbf{bin}(i+1)$; equivalently, the number of trailing zeros in $i+1$.

This gives a $O(n)$ direct successor rule operating entirely on the characteristic vector, without computing rank or unrank explicitly:

> Let $j =$ number of trailing zeros in $\mathbf{rank}(A)+1$.
> Flip bit $j$ in $\chi(A)$ to obtain $\chi(B)$.

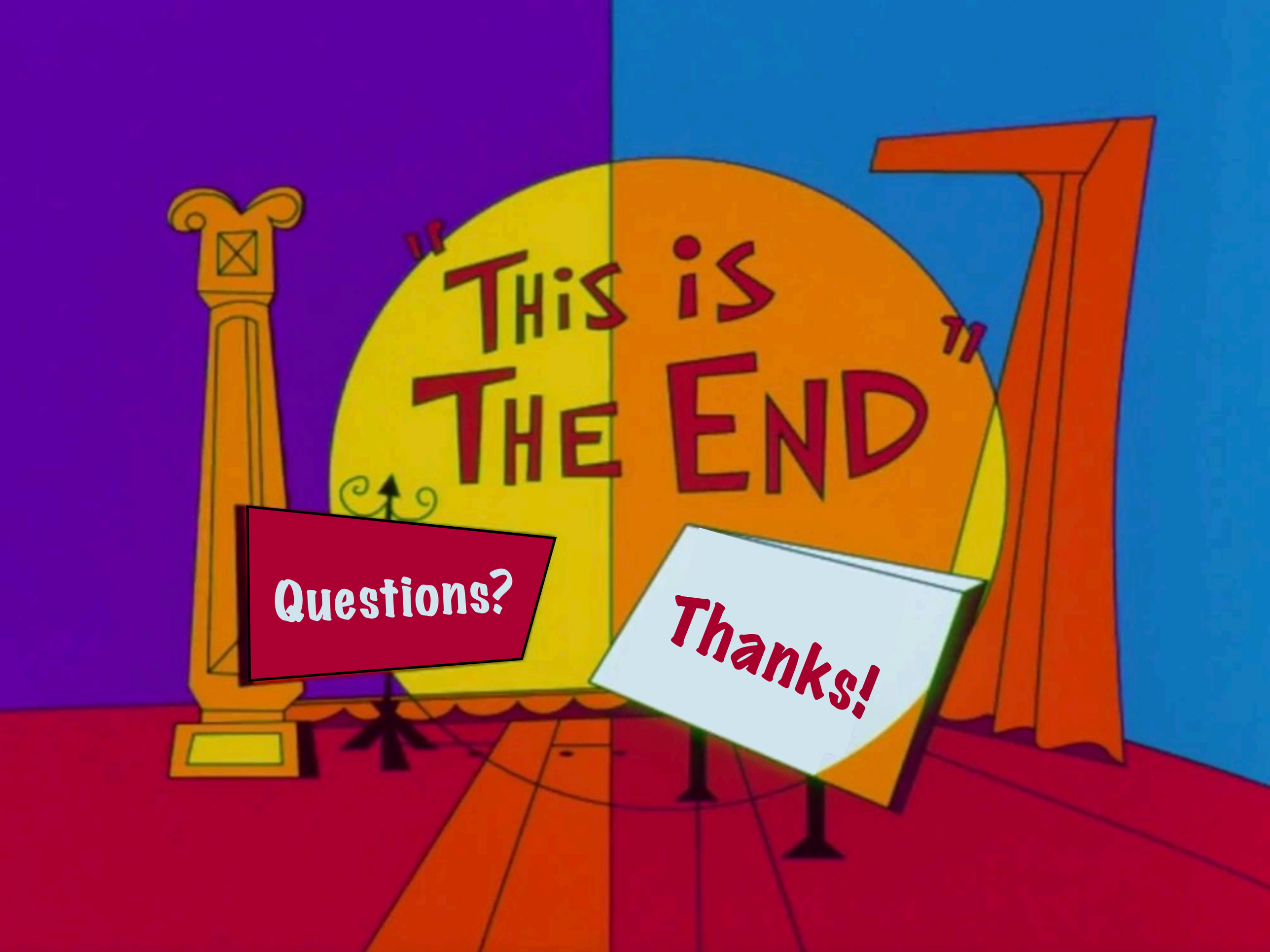
In practice, since computing $\chi(B)$ still requires knowing $\mathbf{rank}(A)$, the fully direct version maintains a parity bit alongside the current subset to determine which bit to flip at each step in $O(1)$ per successor call, thereby amortizing the rank computation across the entire enumeration.

# Summary



Gray Code

- The Hamming distance $\mathrm{dist}(A, B)$ counts how many elements must be added or deleted to go from $A$ to $B$.
- A minimal-change ordering arranges all $2^n$ subsets so that consecutive subsets have Hamming distance exactly 1: each step adds or deletes a single element.
- The characteristic vectors of a minimal-change ordering form a Gray code; equivalently, a Hamiltonian path on the $n$-dimensional hypercube.
- The (binary reflected) Gray code $\mathbf{G}^n$ is constructed inductively.
- The inductive definition of $\mathbf{G}^n$ governs how rank, unrank, and successor are defined: the correctness of each algorithm follows directly from the structure that definition imposes on the sequence.


"This is The End"
Questions?
Thanks!